



MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Inginerie Software și Automatică

RAPORT

Lucrare de laborator Nr. 3

Disciplina: „Tehnici de Programare”

Tema: “Inițializare în python”

Realizat:

St. gr. SI-264, Zlatin Nichita

Verificat:

Reițman Patricia, asist. Univ.

Chișinău 2026

Introducere

Mersul Lucrării:

Partea 1

Lucrăm pe Platforma: <https://www.codedex.io/python>

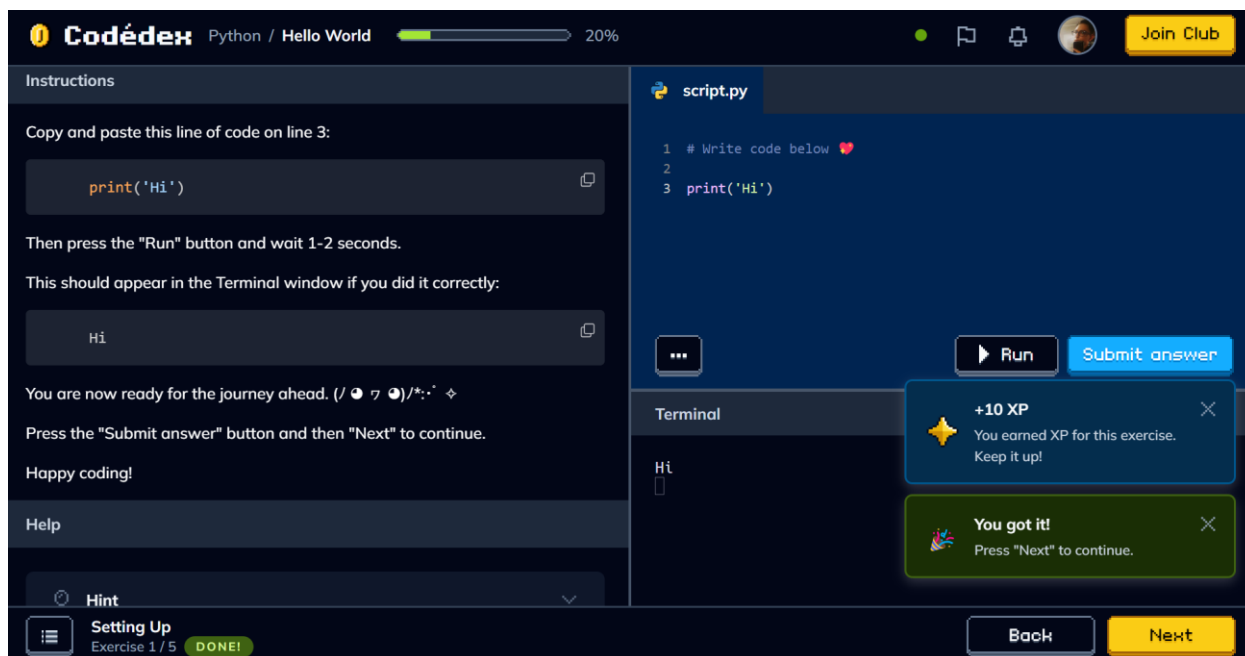


Figura 1.1: Nivelul 1 „Hello World”

Python / Hello World

40%

Join Club

Exercise

+254 also learning

02. Hello World

Print

In Python, the `print()` function is used to tell a computer to "talk." This is something we are going to use *a lot*.



The message we want to display should be inside the parentheses and surrounded by quotes. They can be double quotes `"` or single quotes `'`, but the opening and closing quote marks have to be the same.

This is an example of a `print()` function:

```
print('👋 Howdy')
```

hello_world.py

```
1 # Write code below ❤️
2
3 print("hello_world!")
```

Run

Submit answer

Terminal

```
hello_world!

```

+10 XP

You earned XP for this exercise. Keep it up!

You got it!

Press "Next" to continue.

Back

Next

Hello World

Exercise 2 / 5

DONE!

Figura 1.2: Nivelul 2 „Hello World”

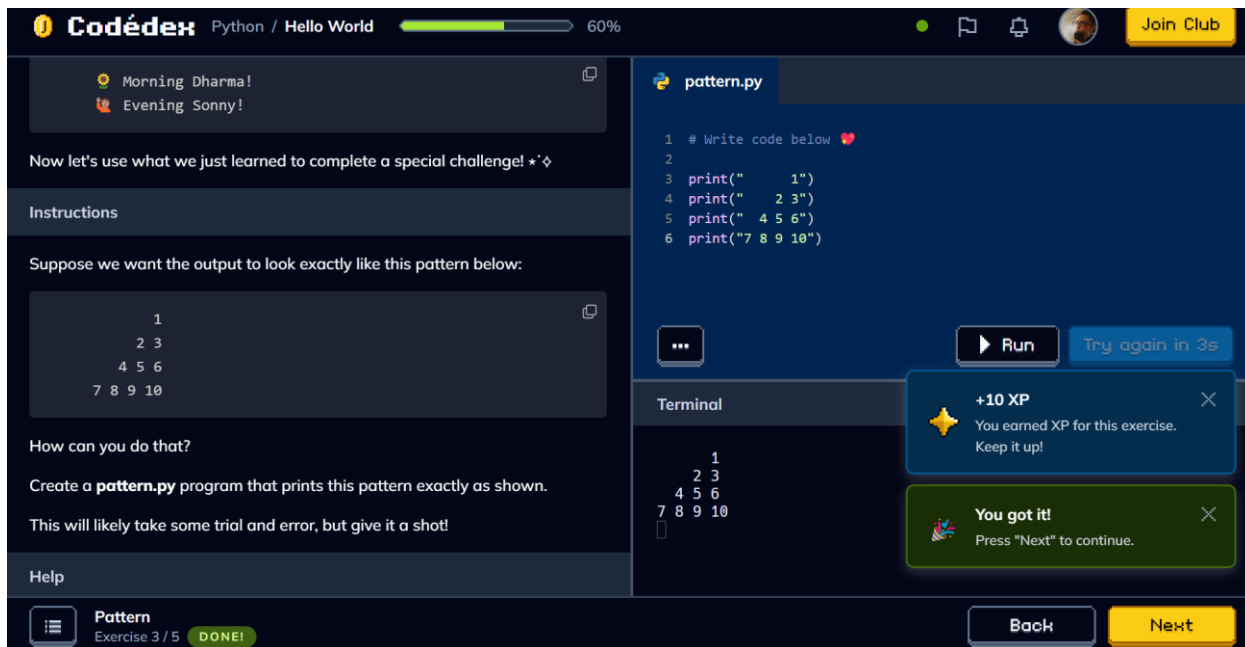


Figura 1.3: Nivelul 3 „Hello World”

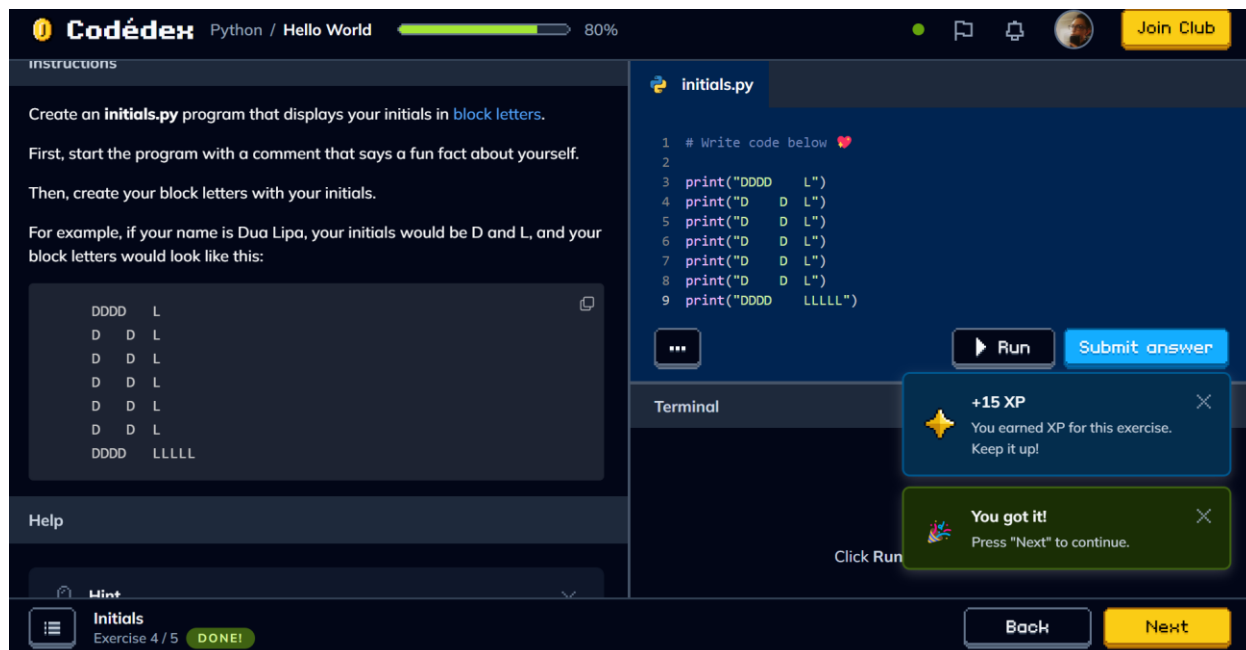


Figura 1.4: Nivelul 4 „Hello World”

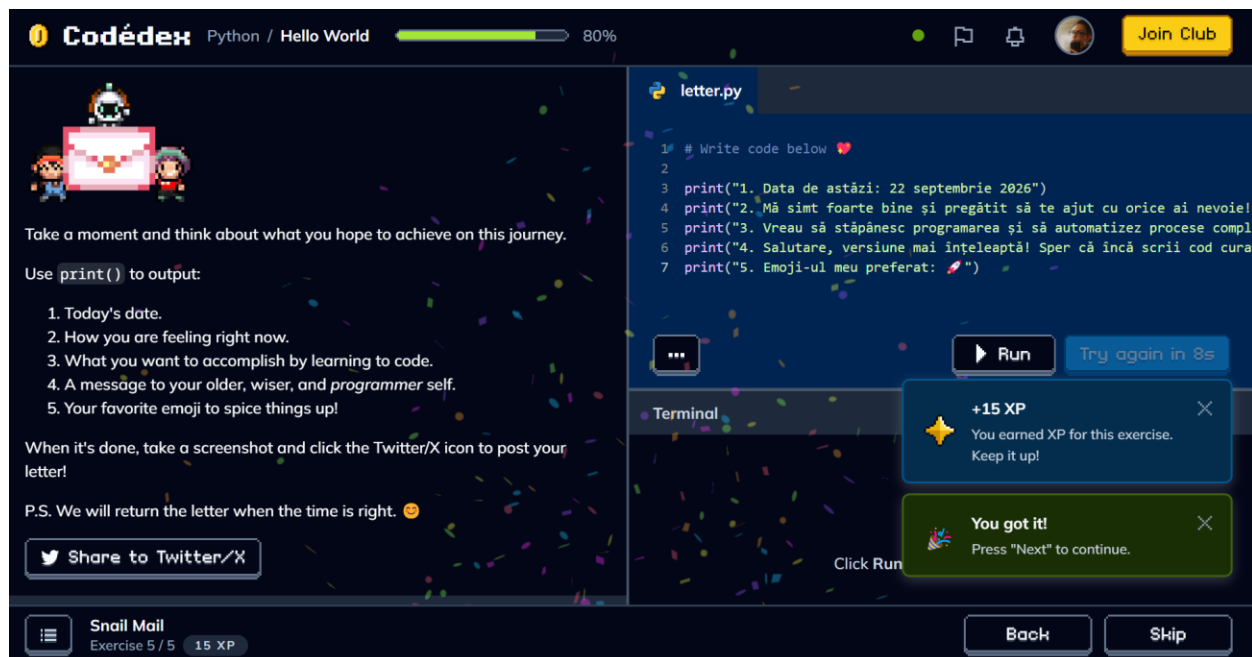
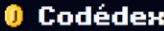


Figura 1.5: Nivelul 5 „Hello World”

 Python / Variables

20%

A Boolean data type, or `bool`, stores a value that can only be either true or false. In Python, it's capitalized `True` or `False`. It's named after the British mathematician [George Boole](#) (1815-1864).

```
late_to_class = False
cranky = True
```

Instructions

No instructions in this exercise! You got a freebie. 🎁

Press "Submit answer" and "Next", and go get 'em!

Optional: Create 4 variables in the code editor.

Help

Hint

 **Data Types**
Exercise 1 / 5 DONE!

Back

Next

script.py

```
1 # Write code below ❤️
2
3 a = 1
4 b = "2"
5 c = 2.0
6 d = True
```

⋮

Run

Submit answer

Terminal

+15 XP
You earned XP for this exercise.
Keep it up!

You got it!
Press "Next" to continue.

Click Run

Figura 1.6: Nivelul 1 „Variables”

Python / Variables

40%

Join Club

Instructions

Create a **temperature.py** program that converts a number from Fahrenheit (°F) to Celsius (°C).

Google the current temperature of **Brooklyn, NY** (where Codédex is based) in °F.

Use the following formula and write it out in Python:

$$Celsius = \frac{(Fahrenheit - 32)}{1.8}$$

Print out the converted temperature. 🛠️

Hint

Temperature
Exercise 2 / 5
DONE!

Back
Next

temperature.py

```

1 # Write code below
2
3 # Setează temperatura în Fahrenheit (exemplu pentru Brooklyn, NY)
4 fahrenheit = 72.0
5
6 # Aplică formula pentru conversia în Celsius
7 celsius = (fahrenheit - 32) / 1.8
8
9 # Afișează rezultatul
10 print("Temperatura în Fahrenheit:", fahrenheit)
11 print("Temperatura în Celsius:", celsius)

```

...
Run
Try again in 4s

Terminal

+15 XP
You earned XP for this exercise. Keep it up!

You got it!
Press "Next" to continue.

Figura 1.7: Nivelul 2 „Variables”

Python / Variables

60%

Join Club

Let's give it a try!

Instructions

The **body mass index (BMI)** was created by a Belgian mathematician in the 1850s and it's used by health and nutrition professionals to quickly estimate body fat in certain populations.

The formula is pretty simple: Take an individual's weight (mass) in kilograms and dividing it by their height in meters, squared.

$$bmi = \frac{mass}{height^2}$$

Create a **bmi.py** program that calculates your own BMI.

Author's note: Psst. BMI is an archaic and oversimplified way to measure personal health. It was created by a mathematician – not a doctor! 💡

Help

BMI
Exercise 3 / 5
DONE!

Back
Next

bmi.py

```

1 # Write code below
2
3 mass = int(input("masa: "))
4 height = int(input("inaltimea: "))
5 bmi = mass / height ** 2

```

...
Run
Try again in 4s

Terminal

+15 XP
You earned XP for this exercise. Keep it up!

You got it!
Press "Next" to continue.

Figura 1.8: Nivelul 3 „Variables”



Figura 1.9: Nivelul 4 „Variables”



Figura 1.10: Nivelul 5 „Variables”

Partea 2

Lucram pe Platforma <https://www.learnpython.org/>

Exercise

Use the "print" function to print the line "Hello, World!".

script.py

1 print("Hello, World!")

IPython Shell

<script.py> output:
Hello, World!

In [1]: |

Great job!

Solution

Submit

This site is generously supported by [DataCamp](#). DataCamp offers online interactive [Python Tutorials](#) for Data Science. Join **over a million** other learners and get started learning Python for data science today!

Figura 2.1: Nivelul „Hello, World!”

Exercise

The target of this exercise is to create a string, an integer, and a floating point number. The string should be named `mystring` and should contain the word "hello". The floating point number should be named `myfloat` and should contain the number 10.0, and the integer should be named `myint` and should contain the number 20.

script.py

```
1 # change this code
2 mystring = "hello"
3 myfloat = 10.0
4 myint = 20
5
6 # testing code
7 if mystring == "hello":
8     print("String: %s" % mystring)
9 if isinstance(myfloat, float) and myfloat == 10.0:
10     print("Float: %f" % myfloat)
11 if isinstance(myint, int) and myint == 20:
12     print("Integer: %d" % myint)
```

IPython Shell

<script.py> output:
String: hello
Float: 10.000000
Integer: 20

In [1]: |

Great job!

Solution

Submit

●

🔗

Figura 2.2: Nivelul „Variables and types”

Exercise

In this exercise, you will need to add numbers and strings to the correct lists using the "append" list method. You must add the numbers 1,2, and 3 to the "numbers" list, and the words 'hello' and 'world' to the strings variable.

You will also have to fill in the variable `second_name` with the second name in the names list, using the brackets operator `[]`. Note that the index is zero-based, so if you want to access the second item in the list, its index will be 1.

script.py

```
1 numbers = []
2 strings = []
3 names = ["John", "Eric", "Jessica"]
4
5 # write your code here
6 numbers.extend([1,2,3])
7 strings.extend(["hello", "world"])
8 second_name = names[1]
9
10 # this code should write out the filled arrays and
11 # the second name in the names list (Eric).
12 print(numbers)
13 print(strings)
14 print("The second name on the names list is %s" %
15       second_name)
```

IPython Shell

<script.py> output:
[1, 2, 3]
['hello', 'world']
The second name on the names list is ['Eric']

<script.py> output:
[1, 2, 3]
['hello', 'world']
The second name on the names list is Eric

In [1]: |

Great Job!

Solution

Submit

●

🔗

Figura 2.3: Nivelul „lists”

Exercise

The target of this exercise is to create two lists called `x_list` and `y_list`, which contain 10 instances of the variables `x` and `y`, respectively. You are also required to create a list called `big_list`, which contains the variables `x` and `y`, 10 times each, by concatenating the two lists you have created.

script.py

```
1 x = object()
2 y = object()
3
4 # TODO: change this code
5 x_list = ["x"] * 10
6 y_list = ["y"] * 10
7 big_list = x_list + y_list
8
9 print("x_list contains %d objects" % len(x_list))
10 print("y_list contains %d objects" % len(y_list))
11 print("big_list contains %d objects" % len(big_list))
12
13 # testing code
```

IPython Shell

```
<script.py> output:
x_list contains 1 objects
y_list contains 1 objects
big_list contains 2 objects

<script.py> output:
x_list contains 1 objects
y_list contains 1 objects
big_list contains 2 objects

<script.py> output:
x_list contains 10 objects
y_list contains 10 objects
big_list contains 20 objects
```

Good work!

Solution

Submit

Figura 2.4: Nivelul „Basic Operators”

Exercise

You will need to write a format string which prints out the data using the following syntax: `Hello John Doe. Your current balance is $53.44.`

script.py

solution.py

```
1 data = ("John", "Doe", 53.44)
2 format_string = "Hello %s %s. Your current balance is %d"
3 print(format_string % data)
```

IPython Shell

```
<script.py> output:
Hello John Doe. Your current balance is 53.

<script.py> output:
Hello John Doe. Your current balance is 53.
Traceback (most recent call last):
  File "script.py", line 3, in <module>
    print("%s %s %s. Your current balance is %d." %
          (format_string, data))
TypeError: not enough arguments for format string

<script.py> output:
Hello John Doe. Your current balance is $53.44.
```

Great work!

Submit

Figura 2.5: Nivelul „String Formatting”

Exercise

Try to fix the code to print out the correct information by changing the string.

script.py

```
17
18 # Convert everything to uppercase
19 print("String in uppercase: %s" % s.upper())
20
21 # Convert everything to lowercase
22 print("String in lowercase: %s" % s.lower())
23
24 # Check how a string starts
25 if s.startswith("Str"):
26     print("String starts with 'Str'. Good!")
27
28 # Check how a string ends
29 if s.endswith("ome!"):
30     print("String ends with 'ome!'. Good!")
31
32 # Split the string into three separate strings,
```

IPython Shell

```
The first occurrence of the letter a = 8
a occurs 2 times
The first five characters are 'Strin'
The next five characters are 'gs ar'
The thirteenth character is 'a'
The characters with odd index are 'tig r
wsm!'
The last five characters are 'some!'
String in uppercase: STRINGS ARE AWESOME!
String in lowercase: strings are awesome!
String starts with 'Str'. Good!
String ends with 'ome!'. Good!
Split the words of the string: ['Strings',
'are', 'awesome!']
```

Great work!

Submit

Figure 2.6: Nivelul „Basic String Operations”

Change the variables in the first section, so that each if statement resolves as True.

script.py

```
6
7 if number > 15:
8     print("1")
9
10 if first_array:
11     print("2")
12
13 if len(second_array) == 2:
14     print("3")
15
16 if len(first_array) + len(second_array) == 5:
17     print("4")
18
19 if first_array and first_array[0] == 1:
20     print("5")
21
```

IPython Shell

```
<script.py> output:
1
2
3
4
5
6

In [1]: |
```

Great Work!

Solution

Submit

Figura 2.7: Nivelul „Conditions”

Exercise

Loop through and print out all even numbers from the numbers list in the same order they are received. Don't print any numbers that come after 237 in the sequence.

script.py

solution.py

```
5 399, 162, 758, 219, 918, 237, 412, 566, 826,
  248, 866, 950, 626, 949, 687, 217,
6 815, 67, 104, 58, 512, 24, 892, 894, 767,
  553, 81, 379, 843, 831, 445, 742, 717,
7 958, 609, 842, 451, 688, 753, 854, 685, 93,
  857, 440, 380, 126, 721, 328, 753, 470,
8 743, 527
9 ]
10
11 # your code goes here
12 for number in numbers:
13     if number == 237:
14         break
15
16     if number % 2 == 1:
17         continue
```

IPython Shell

```
592
236
942
386
462
418
344
236
566
978
328
162
758
918
```

Great work!

Submit

Figura 2.8: Nivelul „Loops”

Exercise

In this exercise you'll use an existing function, and while adding your own to create a fully functional program.

1. Add a function named `list_benefits()` that returns the following list of strings: "More organized code", "More readable code", "Easier code reuse", "Allowing programmers to share and connect code together"
2. Add a function named `build_sentence(info)` which receives a single argument containing a string and returns a sentence starting with the given string and ending with the string " is a benefit of functions!"
3. Run and see all the functions work together!

script.py

IPython Shell

```
1 # Modify this function to return a list of
  strings as defined above
2 def list_benefits():
3     return ["More organized code", "More
  readable code", "Easier code reuse", "Allowing
  programmers to share and connect code together"]
4
5 # Modify this function to concatenate to each
  benefit - " is a benefit of functions!"
6 def build_sentence(benefit):
7     return "%s is a benefit of functions!" %
  benefit
8
9 def name_the_benefits_of_functions():
10     list_of_benefits = list_benefits()
11     for benefit in list_of_benefits:
```

<script.py> output:

```
More organized code is a benefit of functions
More readable code is a benefit of functions
Easier code reuse is a benefit of functions
Allowing programmers to share and connect code
together is a benefit of functions
```

<script.py> output:

```
More organized code is a benefit of functions!
More readable code is a benefit of functions!
Easier code reuse is a benefit of functions!
Allowing programmers to share and connect code
together is a benefit of functions!
```

Nice work!

Solution

Submit

Figura 2.9: Nivelul „Functions”

Exercise

We have a class defined for vehicles. Create two new vehicles called car1 and car2. Set car1 to be a red convertible worth \$60,000.00 with a name of Fer, and car2 to be a blue van named Jump worth \$10,000.00.

script.py

```
4     kind = "car"
5     color = ""
6     value = 100.00
7     def description(self):
8         desc_str = "%s is a %s %s worth $%.2f."
9         % (self.name, self.color, self.kind, self.value)
10        return desc_str
11 # your code goes here
12 car1 = Vehicle()
13 car1.name = "Fer"
14 car1.kind = "car"
15 car1.color = "red"
16 car1.value = 60000.00
17 car2 = Vehicle()
18 car2.name = "Jump"
19 car2.kind = "van"
```

IPython Shell

<script.py> output:
Fer is a red car worth \$60000000.00.
Jump is a blue van worth \$10000000.00.

In [1]: |

Great job!

Solution

Submit

Figura 2.10: Nivelul „Classes and objects”

Exercise

Add "Jake" to the phonebook with the phone number 938273443, and remove Jill from the phonebook.

script.py

```
1 phonebook = {
2     "John" : 938477566,
3     "Jack" : 938377264,
4     "Jill" : 947662781,
5     "Jake" : 938273443
6 }
7 # your code goes here
8 phonebook.pop("Jill")
9 # testing code
10 if "Jake" in phonebook:
11     print("Jake is listed in the phonebook.")
12
13 if "Jill" not in phonebook:
14     print("Jill is not listed in the phonebook.")
```

IPython Shell

<script.py> output:
Jake is listed in the phonebook.
Jill is not listed in the phonebook.

In [1]: |

Nice work!

Solution

Submit

Figura 2.11: Nivelul „Dictionaries”

Exercise

In this exercise, print an alphabetically sorted list of all the functions in the `re` module containing the word `find`.

script.py

solution.py

```
1 import re
2
3 # Your code goes here
4 find_members = []
5 for func in dir(find_members):
6     if "find" in func:
7         find_members.append(func)
8
9 print(find_members)
```

IPython Shell

<script.py> output:
['findall', 'finditer']

In [1]: |

Great work!

Submit

✕

🔍

Figura 2.12: Nivelul „Modules and Packages”

Exercise

Complete the code below so that it reads three integers from a single line, converts each to an `int`, and prints their sum.

script.py

solution.py

```
1 a, b, c = map(int, input().split())
2 print("sum is", a + b + c)
```

IPython Shell

DataCamp encountered the following error:
EOF when reading a line
DataCamp encountered the following error:
EOF when reading a line

In [1]: 1 2 3|

Submit

Restart Session

🔍

Figura 2.13: Nivelul „Input and Output”

will simultaneously switch the values of a and b.

script.py

```
1 # fill in this function
2 def fib():
3     a = 1
4     b = 1
5     yield a
6     yield b
7     #this is a null statement which does nothing
    when executed, useful as a placeholder.
8     for _ in range(20):
9         c = a + b
10        a, b = b, c
11        yield c
12    #this is a null statement which does nothing
    when executed, useful as a placeholder.
13
14 # testing code
15
```

IPython Shell

SyntaxError: invalid syntax

<script.py> output:
Good, The fib function is a generator.
1
1
2
3
5
8
13
21
34
55

Good work!

Solution

Submit

Figura 2.14: Nivelul „Generators”

Exercise

Using a list comprehension, create a new list called "newlist" out of the list "numbers", which contains only the positive numbers from the list, as integers.

script.py

```
1 numbers = [34.6, -203.4, 44.9, 68.3, -12.2, 44.6,
    12.7]
2 newlist = [i for i in numbers if i > 0]
3 print(newlist)
```

IPython Shell

<script.py> output:
[34.6, 44.9, 68.3, 44.6, 12.7]

In [1]:

Very nice!

Solution

Submit

Figura 2.15: Nivelul „List Comprehensions”

Exercise

Write a program using lambda functions to check if a number in the given list is odd. Print "True" if the number is odd or "False" if not for each element.

script.py

```
1 l = [2,4,7,3,14,19]
2 este_par = lambda x: x % 2 == 0
3 for i in l:
4     if este_par(i) is True:
5         print("False")
6     else:
7         print("True")
8
9     # your code here
```

IPython Shell

```
True
True
True
True
True
True
True

<script.py> output:
False
False
True
True
False
True
```

Nice work!

Solution

Submit

Figura 2.16: Nivelul „Lambda Functions”

Exercise

Fill in the `foo` and `bar` functions so they can receive a variable amount of arguments (3 or more). The `foo` function must return the amount of extra arguments received. The `bar` must return `True` if the argument with the keyword `magicnumber` is worth 7, and `False` otherwise.

script.py

```
5     if i:
6         summ += 1
7
8     return summ
9
10 def bar(a, b, c, magicnumber=7):
11     if magicnumber == 7:
12         return True
13     else:
14         return False
15
16
17 # test code
18 if foo(1, 2, 3, 4) == 1:
19     print("Good.")
20 if foo(1, 2, 3, 4, 5) == 2:
```

IPython Shell

```
<script.py> output:
Good.
Better.
Great.
Awesome!

In [1]: |
```

Great work!

Solution

Submit

Figura 2.17: Nivelul „Multiple Function Arguments”

As an example of a "proper" email-matching regex (like the one in the exercise), see [this](#)

script.py

```
10 # Exercise: make a regular expression that will
    match an email
11 def test_email(your_pattern):
12     pattern = re.compile(your_pattern)
13     emails = ["john@example.com", "python
    -list@python.org", "wha.t.`1an?ug{}ly@email
    .com"]
14     for email in emails:
15         if not re.match(pattern, email):
16             print("You failed to match %s" %
    (email))
17         elif not your_pattern:
18             print("Forgot to enter a pattern!")
19         else:
20             print("Pass")
21 pattern = r".{1,30}@email.com" # Your pattern
```

IPython Shell

```
<script.py> output:
<_sre.SRE_Match object; span=(35, 39), match='[on]'\>
None
You failed to match john@example.com
You failed to match python-list@python.org
Pass

<script.py> output:
<_sre.SRE_Match object; span=(35, 39), match='[on]'\>
None
You failed to match john@example.com
You failed to match python-list@python.org
Pass
```

Great work!

Solution

Submit

Figura 2.18: Nivelul „Regular Expressions”

Exercise

Handle all the exception! Think back to the previous lessons to return the last name of the actor.

script.py

```
1 actor = {"name": "John", "last_name": "Cleese",
    "rank": "awesome"}
2
3 # Function to modify!!!
4 def get_last_name():
5     try:
6         return actor["last_name"]
7     except KeyError:
8         print("Error there is no key named
    last_name, just the name")
9
10 # Test code
11 get_last_name()
12 print("All exceptions caught! Good job!")
13 print("The actor's last name is %s" % get_last_name
    ())
```

IPython Shell

```
<script.py> output:
All exceptions caught! Good job!
The actor's last name is Cleese

In [1]: |
```

Great work!

Solution

Submit

Figura 2.19: Nivelul „Exception Handling”

In the exercise below, use the given lists to print out a set containing all the participants from event A which did not attend event B.

script.py

```
1 a = set(["Jake", "John", "Eric"])
2 b = set(["John", "Jill"])
3 print(a.difference(b))
```

IPython Shell

```
Traceback (most recent call last):
  File "script.py", line 3, in <module>
    print(a.difference(b))
AttributeError: 'list' object has no attribute 'difference'

<script.py> output:
{'Jake', 'Eric'}

In [1]: |
```

Nice work!

Solution

Submit

Figura 2.20: Nivelul „Sets”

The aim of this exercise is to print out the JSON string with key-value pair "Me" : 800 added to it.

script.py

solution.py

```
1 import json
2
3 # fix this function, so it adds the given name
4 # and salary pair to salaries_json, and return it
5 def add_employee(salaries_json, name, salary):
6     salaries = json.loads(salaries_json)
7     salaries[name] = salary
8
9     return json.dumps(salaries)
10
11 # test code
12 salaries = '{"Alfred" : 300, "Jane" : 400 }'
13 new_salaries = add_employee(salaries, "Me", 800)
14 decoded_salaries = json.loads(new_salaries)
15 print(decoded_salaries["Alfred"])
16 print(decoded_salaries["Jane"])
```

IPython Shell

```
Traceback (most recent call last):
  File "script.py", line 13, in <module>
    new_salaries = add_employee(salaries, "Me", 800)
  File "script.py", line 7, in add_employee
    test = salaries_json.dumps(salaries_json)
AttributeError: 'str' object has no attribute 'dumps'

<script.py> output:
300
400
800

In [1]: |
```

Great work!

Submit

Figura 2.21: Nivelul „Serialization”

Concluzie:

În urma acestei lucrări de laborator, am asimilat bazele limbajului Python prin intermediul exercițiilor practice pe platformele Codedex.io și Learnpython.org. Am exersat atât concepte fundamentale (variabile, tipuri de date, structuri de control și colecții de date), cât și elemente mai avansate (funcții, programare orientată pe obiecte, expresii regulate și tratarea excepțiilor). Astfel, am obținut o bază practică solidă pentru dezvoltarea și structurarea corectă a codului în Python.